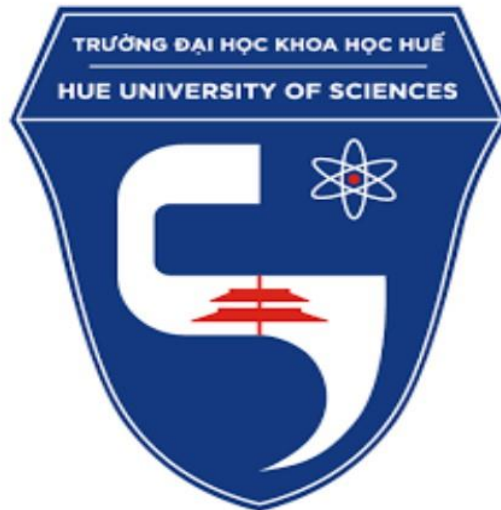


TRƯỜNG ĐẠI HỌC KHOA HỌC HUẾ
KHOA CÔNG NGHỆ THÔNG TIN



BÀI TIỂU LUẬN

HỌC PHẦN: PHÁT TRIỂN ỨNG DỤNG IOT

ĐỀ TÀI

ĐIỀU KHIỂN THIẾT BỊ GIA DỤNG QUA WI-FI VỚI ESP32

Giảng viên hướng dẫn: THS.Võ Việt Dũng

Huế, 04/2026

DANH MỤC VIẾT TẮT

Từ viết tắt	Cụm từ tiếng Anh	Giải nghĩa
API	Application Programming Interface	Giao diện lập trình ứng dụng
ESP32	Espressif 32-bit MCU	Vi điều khiển tích hợp Wifi/Bluetooth
GPIO	General-Purpose Input/Output	Chân vào/ra đa dụng
HTTP	HyperText Transfer Protocol	Giao thức truyền siêu văn bản
IoT	Internet of Things	Mạng lưới vạn vật kết nối
IP	Internet Protocol	Giao thức liên mạng
JSON	JavaScript Object Notation	Định dạng dữ liệu nhẹ
LDR	Light Dependent Resistor	Cảm biến ánh sáng
MQTT	Message Queuing Telemetry Transport	Giao thức truyền thông nhẹ cho IoT
PIR	Passive Infrared Sensor	Cảm biến phát hiện chuyển động
PWM	Pulse Width Modulation	Điều chế độ rộng xung
RH	Relative Humidity	Độ ẩm tương đối
SSID	Service Set Identifier	Tên mạng Wi-Fi
URL	Uniform Resource Locator	Địa chỉ tài nguyên trên web
Wi-Fi	Wireless Fidelity	Mạng không dây

MỤC LỤC

DANH MỤC VIẾT TẮT

LỜI CẢM ƠN	1
1. TỔNG QUAN ĐỀ TÀI.....	2
1.1. Đặt vấn đề	2
1.2. Mục tiêu của đề tài.....	2
1.3. Phạm vi thực hiện	2
2. CƠ SỞ LÝ THUYẾT.....	2
2.1. Vi điều khiển ESP32	2
2.2. Giao thức HTTP trong điều khiển thiết bị	3
2.3. Cảm biến PIR, LDR và dữ liệu mô phỏng.....	3
2.4. Telegram Bot trong hệ thống IoT	4
2.5. Wokwi trong mô phỏng hệ thống nhúng	4
3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	4
3.1. Yêu cầu chức năng.....	4
3.2. Sơ đồ khối hệ thống	5
3.3. Lưu đồ hoạt động	6
3.4. Kịch bản vận hành tiêu biểu	8
4. MÔ PHỎNG PHẦN CỨNG TRÊN WOKWI.....	9
4.1. Danh mục linh kiện.....	9
4.2. Bảng phân bổ chân kết nối.....	10
4.3. Sơ đồ mạch mô phỏng	10
4.4. Giao diện web điều khiển	11
5. THIẾT KẾ PHẦN MỀM VÀ NGUYÊN LÝ TRUYỀN TIN	13
5.1. Kiến trúc chương trình.....	13

5.2. Cơ chế gửi thông báo Telegram.....	13
5.3. Dữ liệu hiển thị trên web	13
5.4. Xử lý ngưỡng và tự động hóa đơn giản	14
6. TRIỂN KHAI TRÊN VS CODE VÀ WOKWI.....	15
6.1. Môi trường thực hiện	15
6.2. Trình tự chạy mô phỏng.....	15
6.3. Đánh giá kết quả mô phỏng	15
7. ĐÁNH GIÁ, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN	16
7.1. Ưu điểm	16
7.2. Hạn chế	16
7.3. Hướng phát triển	16
PHẦN KẾT LUẬN.....	17
TÀI LIỆU THAM KHẢO	
PHỤ LỤC	

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn sâu sắc đến Ban Giám hiệu Trường Đại học Khoa học Huế cùng toàn thể quý thầy cô Khoa Công nghệ Thông tin đã tận tình giảng dạy, truyền đạt cho em những kiến thức quý báu trong suốt quá trình học tập và rèn luyện. Những kiến thức nền tảng và sự chỉ bảo tận tâm của quý thầy cô chính là hành trang quan trọng để em có thể hoàn thành bài tiểu luận này.

Đặc biệt, em xin gửi lời tri ân chân thành đến thầy Võ Việt Dũng – giảng viên phụ trách học phần Phát triển ứng dụng IoT, người đã luôn tận tình hướng dẫn, hỗ trợ và giải đáp những khó khăn mà em gặp phải trong quá trình thực hiện đề tài. Sự quan tâm, những góp ý thiết thực và sự định hướng của Thầy đã giúp em có thêm động lực, từng bước hoàn thiện nội dung cũng như hệ thống của mình.

Bên cạnh đó, em cũng xin chân thành cảm ơn các bạn trong lớp đã luôn sẵn sàng chia sẻ kiến thức, cùng nhau thảo luận và hỗ trợ em trong quá trình thiết kế, mô phỏng mạch và hoàn thiện đề tài. Sự đồng hành và giúp đỡ của các bạn là nguồn động viên rất lớn để em có thể vượt qua những khó khăn trong quá trình thực hiện.

Mặc dù em đã cố gắng để hoàn thành bài tiểu luận một cách tốt nhất, tuy nhiên do thời gian còn hạn chế và kinh nghiệm thực tế của bản thân chưa nhiều nên bài làm khó tránh khỏi những thiếu sót nhất định. Em rất mong nhận được sự thông cảm và những ý kiến đóng góp quý báu từ Thầy và quý thầy cô để em có thể tiếp tục hoàn thiện kiến thức, nâng cao kỹ năng và phát triển đề tài tốt hơn trong tương lai.

Cuối cùng, em xin kính chúc quý thầy cô luôn dồi dào sức khỏe, hạnh phúc và thành công trong sự nghiệp giảng dạy.

Em xin chân thành cảm ơn!

1. TỔNG QUAN ĐỀ TÀI

1.1. Đặt vấn đề

Trong gia đình hiện đại, nhu cầu bật/tắt đèn, quạt từ xa và theo dõi trạng thái môi trường là rất phổ biến. Nếu người dùng phải thao tác hoàn toàn thủ công thì mức độ tiện lợi, linh hoạt và khả năng giám sát sẽ bị hạn chế. Việc sử dụng ESP32 làm trung tâm xử lý, kết hợp web server cục bộ để nhận lệnh HTTP và bot Telegram để xác nhận trạng thái là một mô hình nhỏ gọn nhưng có tính ứng dụng cao trong lĩnh vực smart home.

1.2. Mục tiêu của đề tài

- Xây dựng mô hình điều khiển đèn và quạt bằng ESP32 thông qua giao diện web nội bộ dùng giao thức HTTP.
- Mô phỏng hệ thống trên Wokwi với các phần tử: ESP32 DevKit V1, PIR, LDR, LED, điện trở và dây nối.
- Đọc dữ liệu cảm biến mô phỏng từ PIR và LDR; đồng thời sinh giá trị nhiệt độ, độ ẩm ngẫu nhiên theo khoảng thực tế để hiển thị và cảnh báo.
- Hoàn thiện báo cáo tiểu luận theo đúng hình thức trình bày quy định, có trích dẫn tài liệu tham khảo rõ ràng.

1.3. Phạm vi thực hiện

Mô hình tập trung vào điều khiển trong mạng Wi-Fi nội bộ và kênh thông báo Telegram Bot. Đèn và quạt được biểu diễn bằng hai LED trên sơ đồ Wokwi để thuận lợi cho việc kiểm thử trực quan. Cảm biến PIR và LDR được sử dụng trực tiếp trong mô phỏng; riêng nhiệt độ và độ ẩm được mô phỏng bằng giá trị ngẫu nhiên khi cảm biến tương ứng không được dùng ổn định trong phiên bản chạy cuối. Đề tài chưa phát triển ứng dụng mobile riêng mà ưu tiên web app để dễ triển khai, dễ thử nghiệm và phù hợp.

2. CƠ SỞ LÝ THUYẾT

2.1. Vi điều khiển ESP32

ESP32 là SoC 32-bit do Espressif Systems phát triển, hỗ trợ Wi-Fi 2.4 GHz và Bluetooth, tích hợp nhiều chân GPIO cùng các ngoại vi như ADC, PWM, UART, SPI và I2C. Nhờ khả năng xử

lý tốt và mức giá rẻ, ESP32 thường được dùng trong các ứng dụng IoT gia đình, đo lường, giám sát và điều khiển từ xa. Trong đề tài này, ESP32 vừa đóng vai trò máy chủ web nhỏ để nhận lệnh bật/tắt, vừa làm bộ điều phối cảm biến và gửi thông báo lên Telegram thông qua các yêu cầu HTTP.

So với nhiều dòng vi điều khiển khác, ưu điểm nổi bật của ESP32 là không cần thêm module Wi-Fi rời. Điều này giúp sơ đồ mô phỏng gọn hơn, chương trình dễ tổ chức hơn và chi phí triển khai phần cứng thực tế cũng thấp hơn. Ngoài ra, cộng đồng sử dụng ESP32 đông đảo nên việc tìm thư viện cho web server, HTTP client hay GPIO control rất thuận tiện.

2.2. Giao thức HTTP trong điều khiển thiết bị

HTTP là giao thức truyền tải siêu văn bản được dùng phổ biến trên web. Trong bài toán điều khiển thiết bị gia dụng, ESP32 có thể chạy web server, xuất ra trang HTML đơn giản và nhận các yêu cầu GET như `/light/on`, `/light/off`, `/fan/on` hoặc `/fan/off`. Cách làm này có ưu điểm là không cần cài đặt ứng dụng riêng: người dùng chỉ cần điện thoại hoặc máy tính cùng mạng Wi-Fi, truy cập địa chỉ IP của ESP32 là có thể điều khiển hệ thống.

Do HTTP có cấu trúc rõ ràng, dễ debug bằng trình duyệt, nó phù hợp với mô hình tiểu luận. Khi nhấn nút trên giao diện, trình duyệt gửi yêu cầu tới ESP32. Sau đó chương trình thay đổi trạng thái chân điều khiển LED hoặc quạt, cập nhật giao diện HTML và đồng thời ghi nhận trạng thái mới để phục vụ việc gửi cảnh báo Telegram. Wokwi cũng hỗ trợ mô phỏng mạng cho ESP32, cho phép kiểm tra luồng trong môi trường ảo.

2.3. Cảm biến PIR, LDR và dữ liệu mô phỏng

PIR là cảm biến hồng ngoại thụ động dùng để phát hiện chuyển động, rất phù hợp cho các tình huống cảnh báo ra vào trong mô hình nhà thông minh. Khi cảm biến chuyển sang mức HIGH, ESP32 ghi nhận có chuyển động và có thể phát sinh thông báo Telegram cho người dùng. LDR là điện trở phụ thuộc ánh sáng, cho phép đọc giá trị ADC để ước lượng trạng thái sáng hoặc tối của môi trường.

Trong phiên bản hoàn thiện của đề tài, dữ liệu nhiệt độ và độ ẩm không lấy trực tiếp từ phần cứng mô phỏng mà được sinh ngẫu nhiên trong khoảng hợp lý để đảm bảo hệ thống vận hành ổn định trên Wokwi: đối với thông tin chưa có cảm biến mô phỏng ổn định thì có thể sử dụng giá trị random để kiểm thử luồng hiển thị, cảnh báo và gửi thông báo.

2.4. Telegram Bot trong hệ thống IoT

Telegram Bot API cung cấp giao diện HTTP để ứng dụng bên ngoài gửi tin nhắn, gửi trạng thái hoặc nhận tương tác từ người dùng. Trong mô hình này, bot Telegram đóng vai trò kênh thông báo thời gian gần thực: khi đèn hoặc quạt được bật/tắt, khi nhiệt độ mô phỏng vượt ngưỡng, hoặc khi cảm biến PIR phát hiện chuyển động, ESP32 sẽ gửi một thông điệp tới chat ID đã cấu hình.

Ưu điểm của Telegram là tạo bot nhanh, miễn phí, không cần tự xây dựng máy chủ thông báo riêng và có thể sử dụng ngay trên điện thoại. So với việc chỉ hiển thị trạng thái trên web nội bộ, Telegram giúp người dùng theo dõi thiết bị từ xa thuận tiện hơn, đặc biệt khi muốn nhận cảnh báo mà không cần luôn mở trang điều khiển.

2.5. Wokwi trong mô phỏng hệ thống nhúng

Wokwi là nền tảng mô phỏng trực tuyến cho Arduino, ESP32 và nhiều loại linh kiện điện tử. Công cụ này cho phép thiết kế sơ đồ nối dây, chạy chương trình, quan sát Serial Monitor và kiểm thử các giao tiếp như Wi-Fi, HTTP hoặc MQTT đối với ESP32 [4], [5]. Trong phạm vi bài tiểu luận, Wokwi giúp minh họa trực quan mạch điện, rút ngắn thời gian lắp ráp và thuận lợi khi nộp báo cáo.

3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

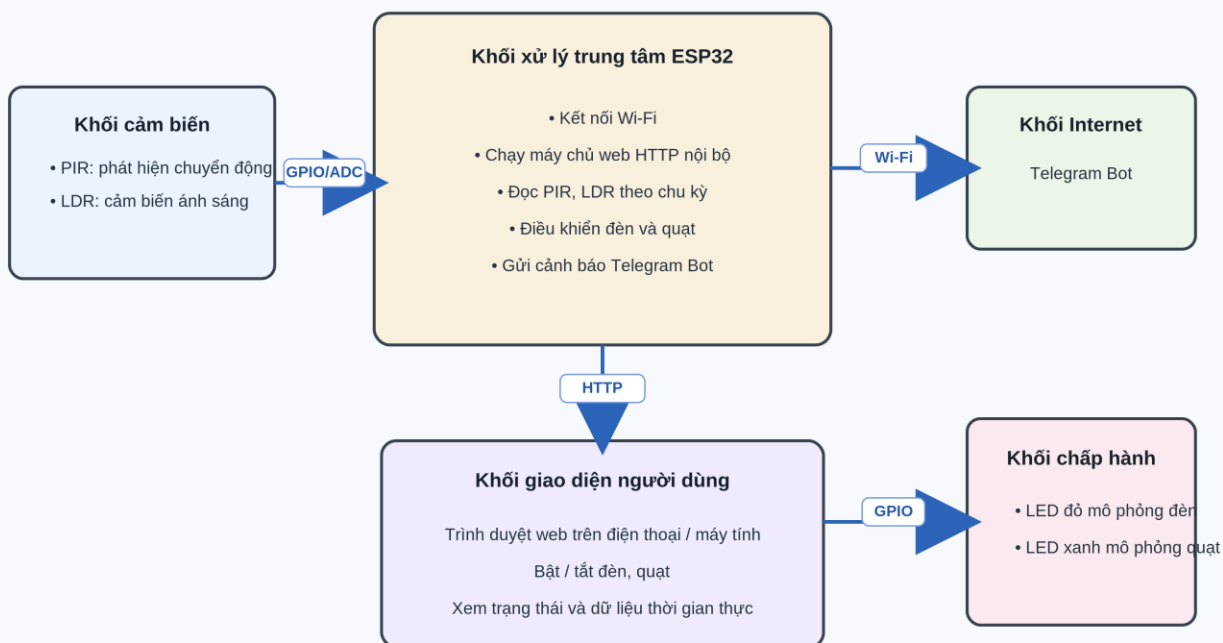
3.1. Yêu cầu chức năng

Mã chức năng	Nội dung	Tác động	Kết quả mong muốn
F1	Bật/tắt đèn qua web	GPIO điều khiển LED	Đèn đổi trạng thái đúng theo nút nhấn
F2	Bật/ tắt quạt qua web	GPIO/PWM điều khiển quạt	Quạt mô phỏng chạy hoặc dừng
F3	Đọc PIR, LDR và sinh dữ liệu nhiệt – ẩm mô phỏng	Thu thập dữ liệu môi trường	Thông số hiển thị đúng trên web
F4	Gửi thông báo Telegram	HTTP GET tới Bot API	Người dùng nhận được tin nhắn
F5	Mô phỏng trên Wokwi	Kiểm thử toàn bộ luồng xử lý	Web, wokwi và Telegram hoạt động ổn định

3.2. Sơ đồ khối hệ thống

Hệ thống được tổ chức theo bốn khối chính: khối thu thập dữ liệu, khối xử lý trung tâm, khối giao diện điều khiển và khối thông báo từ xa. PIR và LDR cung cấp dữ liệu trực tiếp cho ESP32; nhiệt độ và độ ẩm được tạo ngẫu nhiên trong mã chương trình để phục vụ hiển thị và cảnh báo. ESP32 xử lý các yêu cầu HTTP, đổi trạng thái GPIO để bật/tắt LED mô phỏng đèn và quạt, sau đó đưa thông tin lên giao diện web và xếp hàng tin nhắn Telegram khi có sự kiện đáng chú ý.

Sơ đồ khối hệ thống điều khiển thiết bị gia dụng qua Wi-Fi với ESP32

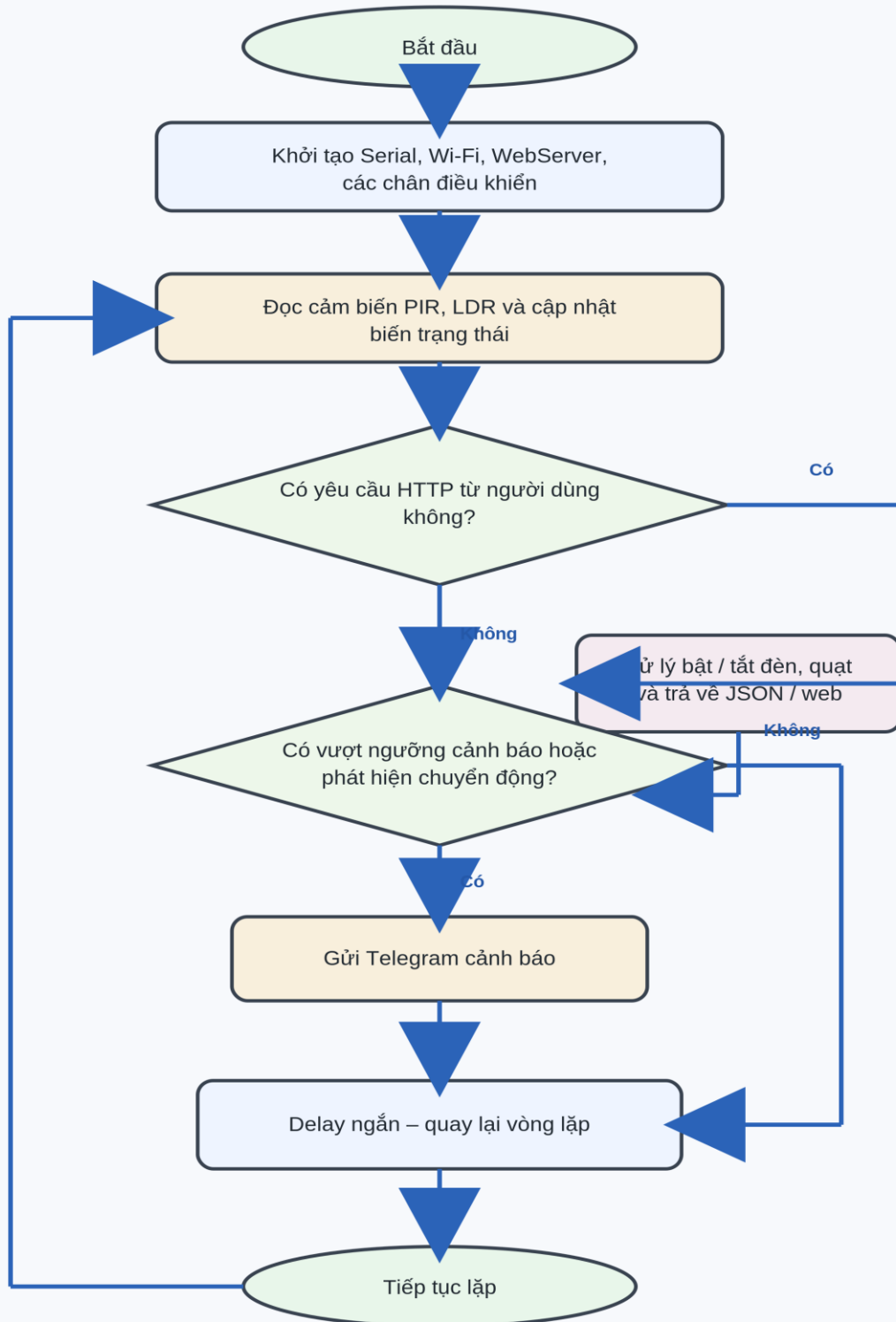


Hình 3.1. Sơ đồ khối của hệ thống điều khiển thiết bị gia dụng qua Wi-Fi với ESP32

3.3. Lưu đồ hoạt động

Khi khởi động, ESP32 kết nối Wi-Fi, khởi tạo web server và thiết lập trạng thái ban đầu cho các GPIO. Trong vòng lặp chính, chương trình ưu tiên xử lý các yêu cầu HTTP từ người dùng, sau đó định kỳ đọc PIR, LDR và sinh dữ liệu nhiệt độ/độ ẩm ngẫu nhiên để cập nhật giao diện. Mỗi thay đổi trạng thái như bật đèn, tắt quạt hay phát hiện chuyển động đều được xếp vào hàng đợi Telegram để gửi sau, nhờ đó giao diện web vẫn phản hồi nhanh mà vẫn bảo đảm có thông báo xác nhận.

Lưu đồ hoạt động chương trình



Hình 3.2. Lưu đồ hoạt động của chương trình điều khiển

3.4. Kịch bản vận hành tiêu biểu

Kịch bản thứ nhất là người dùng mở trang web trên điện thoại hoặc máy tính, nhấn nút bật đèn. ESP32 nhận yêu cầu `/api/led/on`, đổi chân GPIO2 sang mức cao, LED đỏ trên Wokwi sáng lên, giao diện cập nhật lại và bot Telegram gửi thông báo Đèn LED đã bật. Kịch bản thứ hai là người dùng bật quạt bằng nút `/api/fan/on`; ESP32 đặt GPIO15 mức cao, LED xanh đại diện cho quạt sáng lên và Telegram gửi xác nhận. Kịch bản thứ ba là PIR phát hiện chuyển động hoặc nhiệt độ mô phỏng vượt ngưỡng; chương trình lập tức xếp hàng thông báo cảnh báo để gửi tới Telegram.

4. MÔ PHỎNG PHẦN CỨNG TRÊN WOKWI

4.1. Danh mục linh kiện

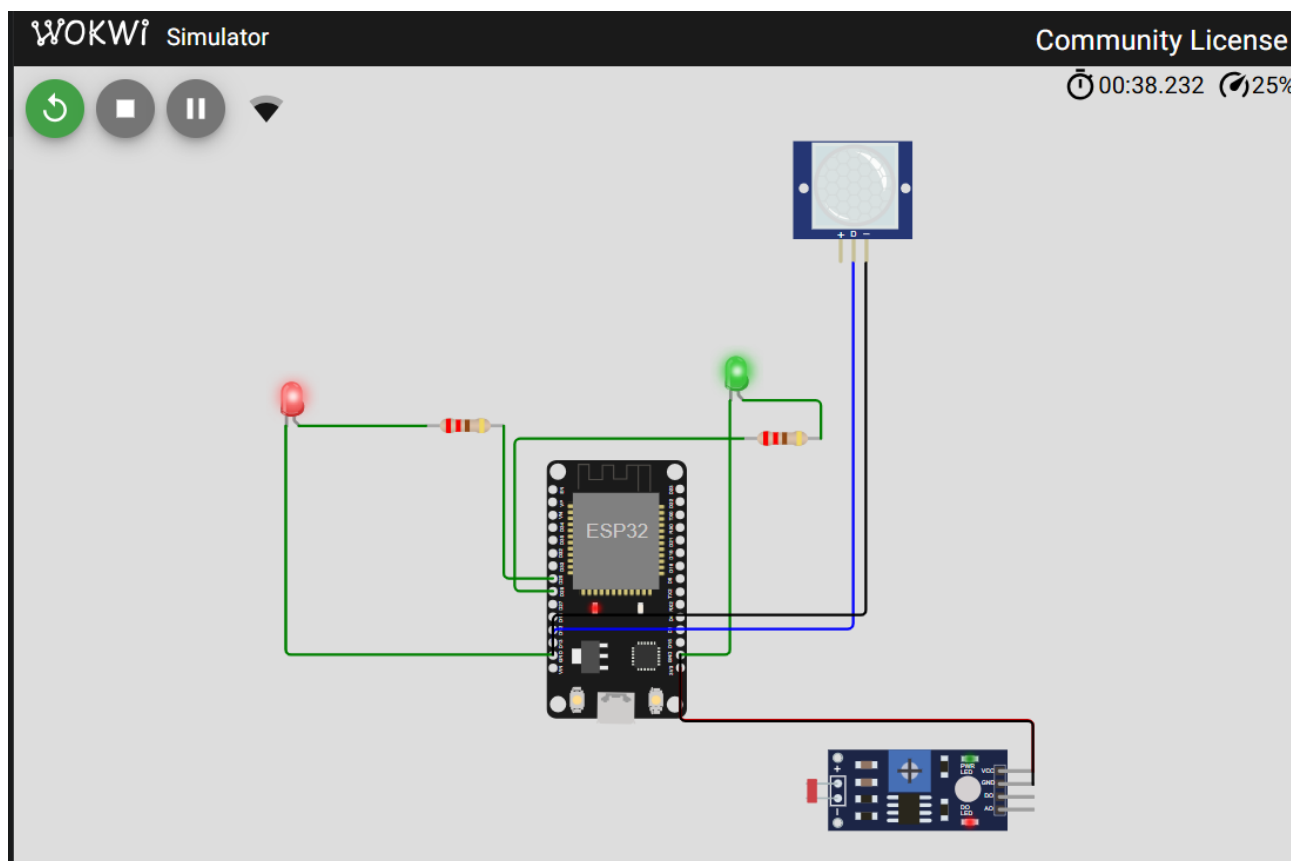
STT	Linh kiện	Số lượng	Mục đích sử dụng
1	ESP32 DevKit V1	1	Bộ mạch xử lý trung tâm
2	PIR motion sensor	1	Phát hiện chuyển động
3	LDR module	1	Đo mức ánh sáng tương đối
4	LED đỏ	1	Mô phỏng thiết bị đèn
5	LED xanh	1	Mô phỏng thiết bị quạt
6	Điện trở 220Ω	2	Hạn dòng cho hai LED
7	Dây nối	Nhiều	Liên kết các phân tử trong sơ đồ
8	Dữ liệu nhiệt độ, độ ẩm mô phỏng	-	Sinh ngẫu nhiên trong chương trình

4.2. Bảng phân bổ chân kết nối

Thiết bị	Chân tín hiệu	Chân ESP32	Ghi chú
PIR	OUT	GPIO13	Phát hiện chuyển động
LDR	AO	GPIO34	Đọc giá trị ADC
LED đèn	A	GPIO2	Điều khiển đèn mô phỏng
LED quạt	A	GPIO15	Điều khiển quạt mô phỏng
Nhiệt độ, độ ẩm mô phỏng	-	-	Sinh giá trị ngẫu nhiên trong mã chương trình

4.3. Sơ đồ mạch mô phỏng

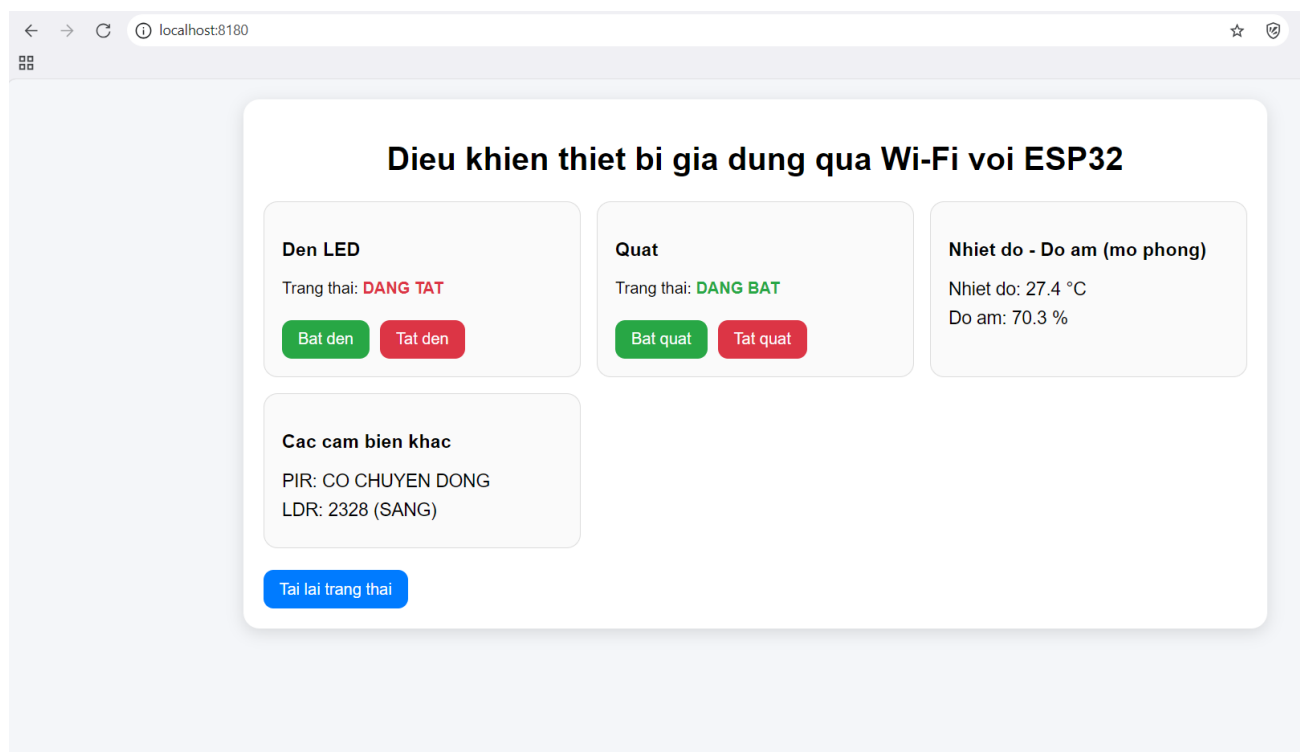
Sơ đồ Wokwi của phiên bản chạy cuối sử dụng ESP32 ở vị trí trung tâm, hai LED để mô phỏng hai thiết bị gia dụng, cảm biến PIR để phát hiện chuyển động và module LDR để đọc mức sáng. Mạch được thiết kế đơn giản để dễ trình bày, nhưng vẫn thể hiện đầy đủ luồng xử lý của bài toán điều khiển thiết bị gia dụng qua Wi-Fi.



Hình 4.1. Sơ đồ nối dây mô phỏng trên Wokwi (phiên bản chạy cuối) khi bật đèn và quạt

4.4. Giao diện web điều khiển

Giao diện web được thiết kế theo dạng bảng điều khiển tối giản, hiển thị rõ trạng thái đèn, quạt, giá trị nhiệt độ – độ ẩm mô phỏng, mức sáng từ LDR và trạng thái chuyển động từ PIR. Các nút bấm gọi trực tiếp các endpoint HTTP như `/api/led/on`, `/api/led/off`, `/api/fan/on` và `/api/fan/off`. Giao diện dùng JavaScript để nạp lại trạng thái định kỳ, vì vậy sau mỗi thao tác người dùng có thể quan sát được sự thay đổi gần như ngay lập tức mà không cần tải lại toàn bộ trang.



Hình 4.2. Giao diện web điều khiển trên trình duyệt (localhost:8180)

5. THIẾT KẾ PHẦN MỀM VÀ NGUYÊN LÝ TRUYỀN TIN

5.1. Kiến trúc chương trình

Chương trình được chia thành các phần rõ ràng gồm: cấu hình phần cứng, kết nối Wi-Fi, khởi tạo web server, đọc PIR và LDR, sinh giá trị nhiệt độ – độ ẩm mô phỏng, tạo giao diện HTML và hàm gửi Telegram. Trong hàm loop, chương trình ưu tiên xử lý web server liên tục để thao tác điều khiển không bị trễ quá nhiều, đồng thời chỉ gửi Telegram khi hàng đợi còn bản tin và hệ thống không bận xử lý yêu cầu web vừa phát sinh.

5.2. Cơ chế gửi thông báo Telegram

Để gửi tin nhắn Telegram, ESP32 tạo một yêu cầu HTTP GET tới địa chỉ dạng `https://api.telegram.org/bot<TOKEN>/sendMessage?chat_id=<CHAT_ID>&text=<NOI_DUNG>`. Nội dung được mã hóa URL ở mức cơ bản để tránh lỗi khi có khoảng trắng hoặc ký tự đặc biệt. Trong triển khai cuối, tin nhắn không gửi trực tiếp ngay trong hàm xử lý nút bấm mà được đưa vào hàng đợi, sau đó vòng lặp chính gửi ra ngoài sau một khoảng nghỉ ngắn. Cách làm này giúp giao diện web giảm cảm giác bị đơ khi người dùng bật/tắt thiết bị liên tiếp [6].

Trong mô hình hiện tại, bot được dùng theo ba nhóm tình huống. Nhóm thứ nhất là thông báo thao tác chủ động: bật đèn, tắt đèn, bật quạt, tắt quạt. Nhóm thứ hai là cảnh báo theo sự kiện: phát hiện chuyển động hoặc nhiệt độ mô phỏng vượt ngưỡng. Nhóm thứ ba là thông báo khởi động, dùng để xác nhận hệ thống đã kết nối Wi-Fi và sẵn sàng nhận lệnh điều khiển.

5.3. Dữ liệu hiển thị trên web

Trang web của ESP32 hiển thị bốn nhóm thông tin: trạng thái đèn, trạng thái quạt, nhiệt độ – độ ẩm mô phỏng, và các giá trị PIR/LDR. Mỗi 2 đến 4 giây, JavaScript gọi endpoint `/status` để lấy JSON mới nhất và cập nhật giao diện. Cơ chế này giúp trang điều khiển luôn phản ánh trạng thái hiện tại của hệ thống mà không cần phụ thuộc vào Serial Monitor.

5.4. Xử lý ngưỡng và tự động hóa đơn giản

Trong phiên bản hoàn thiện, hệ thống chưa kích hoạt chế độ tự bật quạt hoặc đèn một cách bắt buộc, nhưng đã có cơ chế kiểm tra ngưỡng rõ ràng. Nếu nhiệt độ mô phỏng vượt 35°C , chương trình tạo cảnh báo Telegram. Nếu PIR phát hiện chuyển động, chương trình cũng gửi thông báo để người dùng biết có hoạt động bất thường. Đây là bước trung gian hợp lý trước khi mở rộng sang điều khiển tự động hoàn toàn.

6. TRIỂN KHAI TRÊN VS CODE VÀ WOKWI

6.1. Môi trường thực hiện

Mã nguồn được phát triển bằng VS Code với PlatformIO vì công cụ này hỗ trợ quản lý thư viện, biên dịch và nạp chương trình cho ESP32 khá tốt. Khi cài tiện ích Wokwi cho VS Code, người dùng có thể mô phỏng trực tiếp dự án PlatformIO, quan sát Serial Monitor và kiểm thử giao diện web. Với đề tài sinh viên, đây là lựa chọn phù hợp vì dễ trình bày và có thể chuyển từ mô phỏng sang phần cứng thật về sau.

6.2. Trình tự chạy mô phỏng

- Cài VS Code, PlatformIO IDE và Wokwi for VS Code.
- Tạo dự án ESP32; cấu hình platformio.ini theo board ESP32 DevKit V1.
- Không cần cài thư viện DHT trong phiên bản chạy cuối; chương trình chính sử dụng các thư viện WiFi, WebServer, WiFiClientSecure và HTTPClient tích hợp sẵn trong framework Arduino của ESP32.
- Chuẩn bị diagram.json hoặc dự án Wokwi tương ứng để mô phỏng sơ đồ nối dây.
- Chạy mô phỏng, bật Private Gateway trong Wokwi nếu cần, sau đó truy cập địa chỉ localhost:8180 để điều khiển bằng trình duyệt.
- Kiểm tra ứng dụng Telegram để xác nhận bot nhận được thông báo trạng thái.

6.3. Đánh giá kết quả mô phỏng

Kết quả mô phỏng cho thấy hệ thống đáp ứng đúng các lệnh bật/tắt đèn và quạt; LED tương ứng trên Wokwi đổi trạng thái đúng theo giao diện web. PIR và LDR được đọc trực tiếp từ mô phỏng; trong khi đó nhiệt độ và độ ẩm mô phỏng hiển thị ổn định trên web, đủ để kiểm thử cơ chế cảnh báo ngưỡng. Telegram nhận được thông báo ở các sự kiện quan trọng như bật/tắt thiết bị, cảnh báo chuyển động và cảnh báo nhiệt độ cao.

7. ĐÁNH GIÁ, HẠN CHẾ VÀ HƯỚNG PHÁT TRIỂN

7.1. Ưu điểm

Mô hình tận dụng tốt các thế mạnh của ESP32: có Wi-Fi tích hợp, đủ chân GPIO, chi phí thấp và dễ mở rộng. Web app cục bộ giúp điều khiển nhanh mà không cần cài ứng dụng riêng. Telegram tạo thêm một lớp thông báo từ xa hữu ích, làm cho đề tài gần với ứng dụng nhà thông minh thực tế hơn. Wokwi giúp minh họa trực quan, thuận lợi cho việc trình bày và kiểm thử lại.

7.2. Hạn chế

Do là mô hình tiểu luận nên hệ thống mới tập trung vào lớp điều khiển và thông báo, chưa tích hợp xác thực người dùng hoặc cơ chế bảo mật đầy đủ. Việc sử dụng nhiệt độ – độ ẩm mô phỏng làm cho hệ thống phù hợp mục tiêu học tập và kiểm thử luồng xử lý, nhưng chưa thể thay thế hoàn toàn dữ liệu cảm biến thực khi triển khai ngoài đời.

7.3. Hướng phát triển

Trong tương lai, đề tài có thể được mở rộng theo các hướng: thay hai LED mô phỏng bằng relay để điều khiển đèn/quạt thật; tích hợp thêm cảm biến nhiệt độ – độ ẩm ổn định hơn trên phần cứng thật; bổ sung giao thức MQTT hoặc Blynk để giám sát qua Internet; lưu dữ liệu lên đám mây và xây dựng app di động riêng. Một hướng khác là thêm cơ chế xác thực cho web app để tăng độ an toàn khi triển khai ngoài thực tế.

PHẦN KẾT LUẬN

Bài tiểu luận đã xây dựng được một mô hình điều khiển thiết bị gia dụng qua Wi-Fi với ESP32. Hệ thống cho phép bật/tắt đèn và quạt qua giao diện web, đọc dữ liệu PIR và LDR từ Wokwi, mô phỏng nhiệt độ – độ ẩm bằng giá trị ngẫu nhiên trong khoảng thực tế và gửi thông báo lên Telegram. Nhờ đó, mô hình đáp ứng đúng yêu cầu môn học đồng thời thể hiện được tư duy thiết kế một ứng dụng IoT hoàn chỉnh từ phần cứng, truyền thông đến giao diện sử dụng.

Về mặt học thuật, bài làm cho thấy ESP32 là nền tảng phù hợp cho các bài toán smart home cơ bản. Về mặt thực hành, Wokwi và VS Code giúp rút ngắn quá trình triển khai, thử nghiệm và hiệu chỉnh. Từ kết quả đạt được, có thể khẳng định đề tài có thể tiếp tục phát triển thành mô hình thực nghiệm có tính ứng dụng cao hơn trong các học phần IoT sau này.

TÀI LIỆU THAM KHẢO

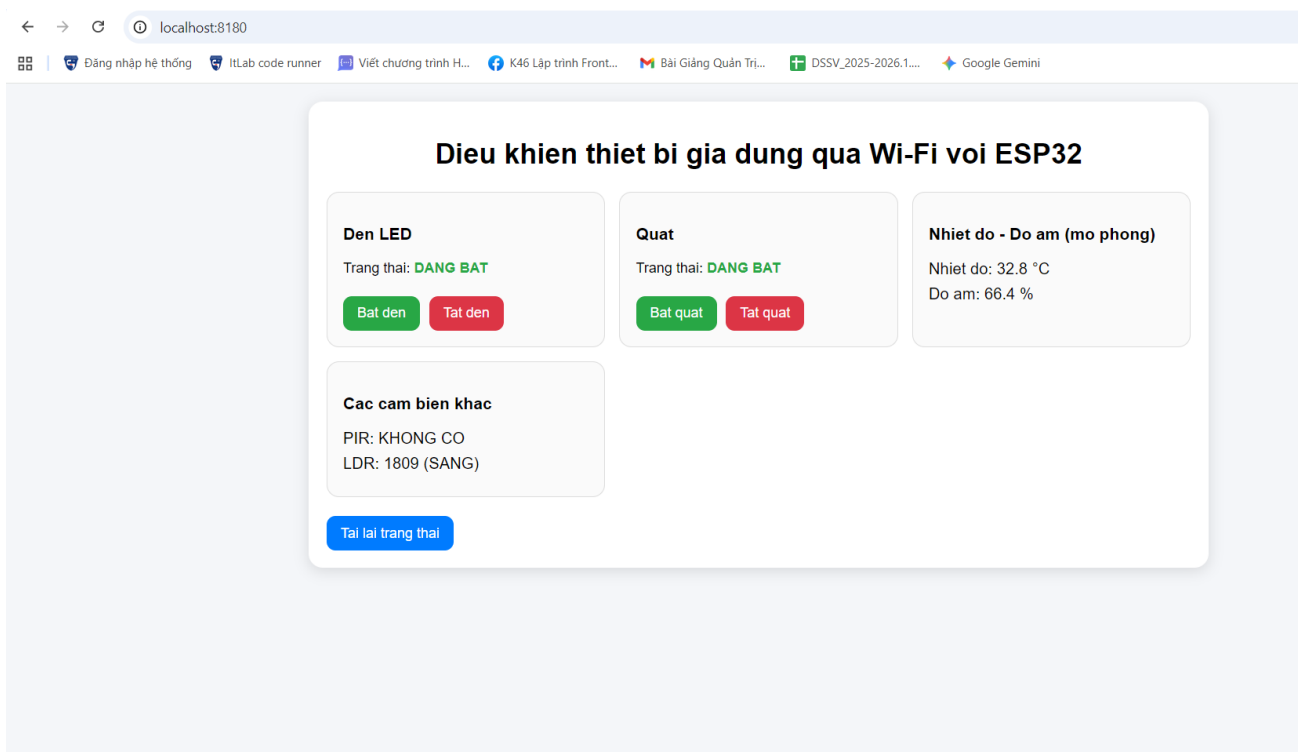
Tiếng Việt:

- [1] Nguyễn Tấn Phước (2021). *Giáo trình Vi điều khiển ESP32 và ứng dụng IoT*, Nxb Đại học Quốc gia TP.HCM, TP. Hồ Chí Minh.
- [2] Phạm Quang Huy, Lê Cảnh Trung (2019). *Thực hành Internet of Things với NodeMCU ESP8266 và ESP32*, Nxb Thanh Niên, Hà Nội.

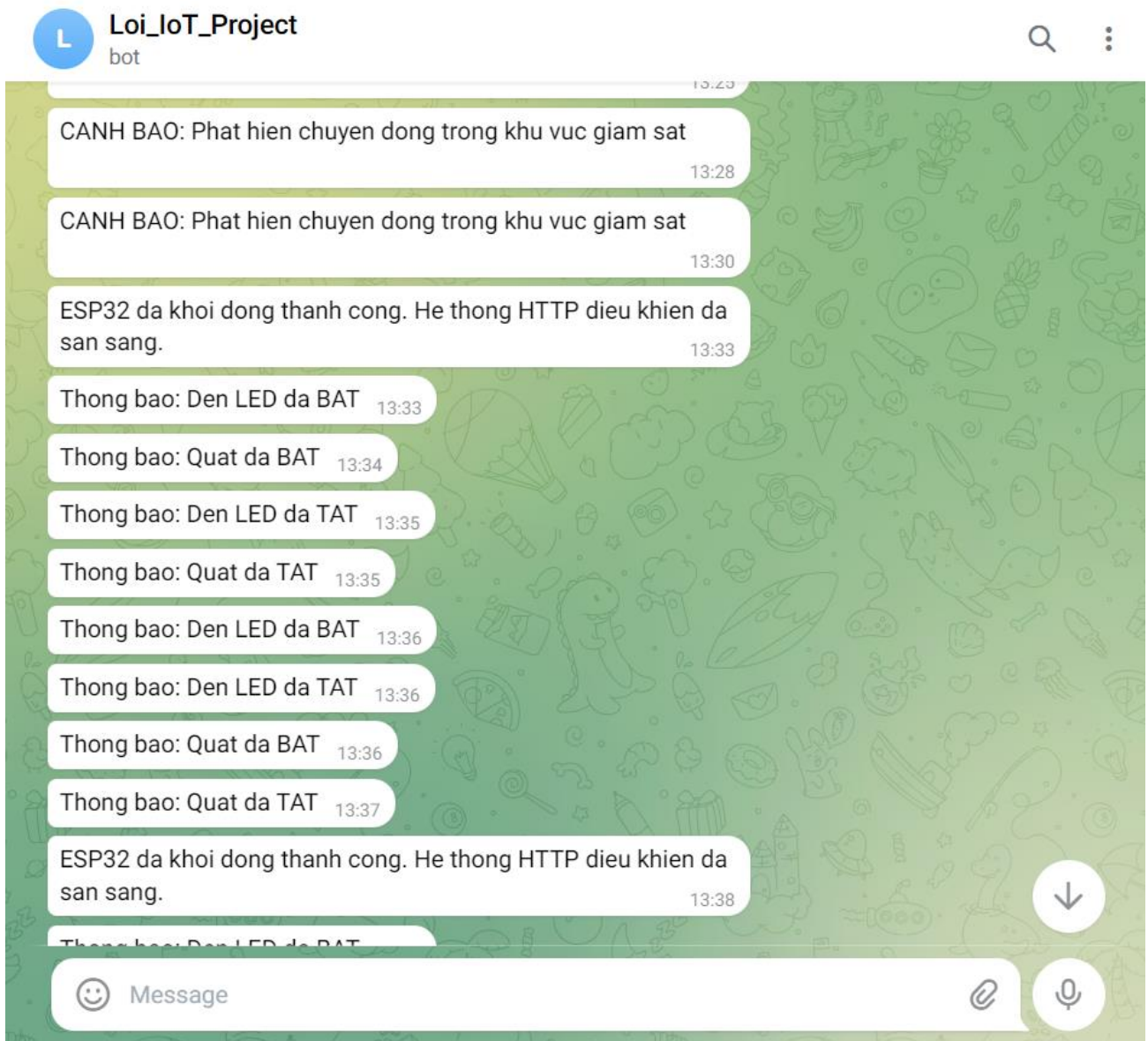
Tiếng Anh:

- [3] Espressif Systems (2024). *ESP32 Series Datasheet*. Truy cập ngày 02/04/2026.
- [4] Telegram (2026). *Telegram Bot API*. Truy cập ngày 03/04/2026.
- [5] Wokwi Docs (2026). *ESP32 Simulation*. Truy cập ngày 02/04/2026.
- [6] Wokwi Docs (2026). *ESP32 WiFi Networking*. Truy cập ngày 02/04/2026.
- [7] PlatformIO (2026). *Library Management*. Truy cập ngày 05/04/2026.

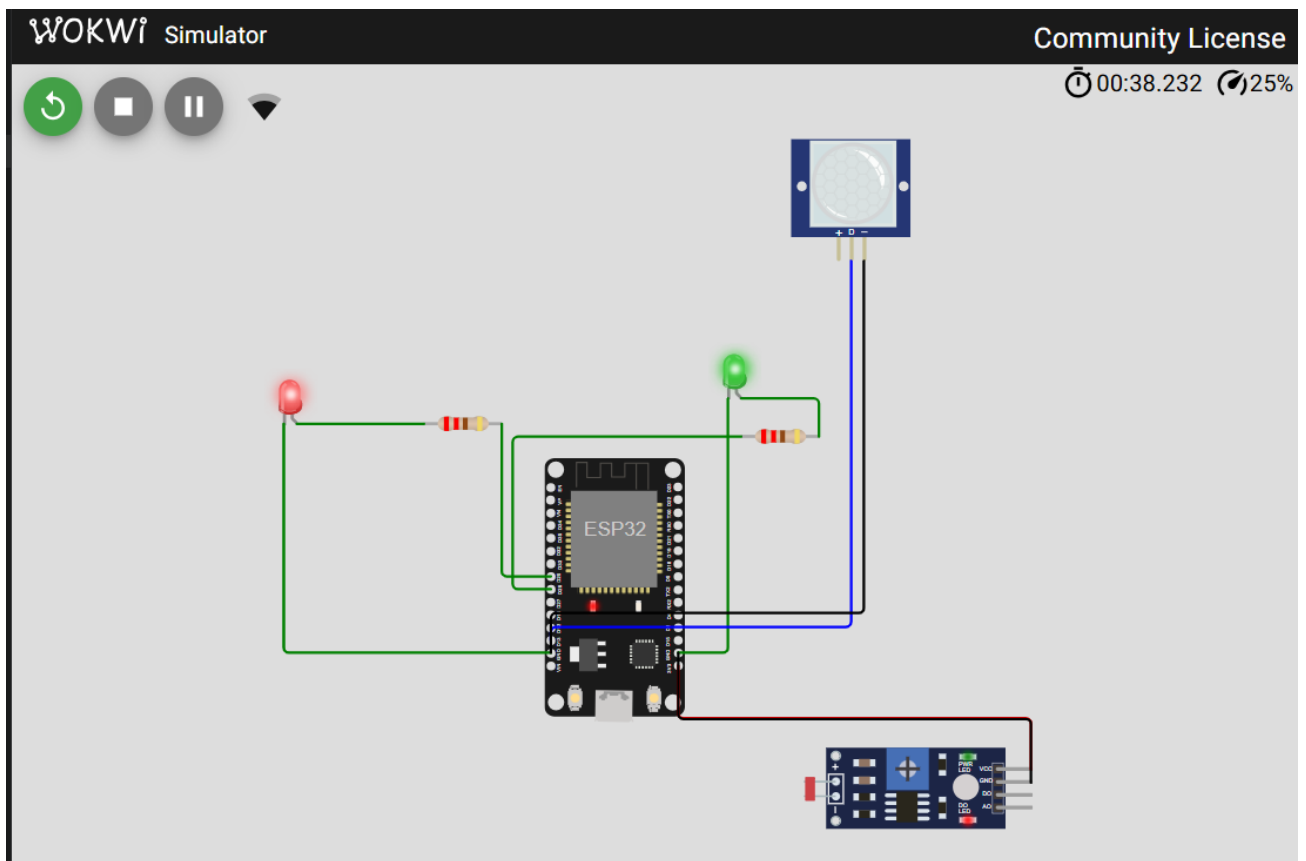
PHỤ LỤC: KẾT QUẢ MÔ PHỎNG VÀ MÃ NGUỒN



Hình. Thao tác chức năng trên trình duyệt localhost:8180



Hình. Thông báo từ Telegram bot



Hình. Kết quả chạy mô phỏng trên wokwi

[Mã nguồn]: https://github.com/vvdung-husc/2025-2026.2.TIN4024.001/tree/main/TEAM_11/DaoVanLoi/BaiTieuLuan/MaNguonBaiTieuLuan/ESP32_TieuLuan

Mã nguồn các file chính:

file main.cpp:

```
#include <WiFi.h>
#include <WebServer.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include "config.h"
```

```

WebServer server(80);

bool ledState = false;
bool fanState = false;

float temperatureC = 27.0;
float humidity = 60.0;
int ldrValue = 0;
int pirValue = 0;

unsigned long lastSensorRead = 0;
unsigned long lastTempAlertTime = 0;
unsigned long lastMotionAlertTime = 0;
unsigned long lastHttpActionTime = 0;

const unsigned long sensorInterval = 2500;

// Hang doi Telegram
const int TELEGRAM_QUEUE_SIZE = 10;
String telegramQueue[TELEGRAM_QUEUE_SIZE];
int telegramHead = 0;
int telegramTail = 0;
int telegramCount = 0;

unsigned long lastTelegramSend = 0;
const unsigned long telegramGapMs = 4000;
const unsigned long httpQuietBeforeTelegramMs = 3000;

String urlEncode(const String &str) {

```

```

String encoded = "";
char c;
char code0;
char code1;

for (int i = 0; i < str.length(); i++) {
    c = str.charAt(i);

    if (isalnum((unsigned char)c)) {
        encoded += c;
    } else if (c == ' ') {
        encoded += "%20";
    } else if (c == '\n') {
        encoded += "%0A";
    } else {
        code1 = (c & 0x0f) + '0';
        if ((c & 0x0f) > 9) code1 = (c & 0x0f) - 10 + 'A';
        c = (c >> 4) & 0x0f;
        code0 = c + '0';
        if (c > 9) code0 = c - 10 + 'A';
        encoded += '%';
        encoded += code0;
        encoded += code1;
    }
}

return encoded;
}

void enqueueTelegramMessage(const String &msg) {
    if (telegramCount >= TELEGRAM_QUEUE_SIZE) {

```

```

    telegramHead = (telegramHead + 1) % TELEGRAM_QUEUE_SIZE;
    telegramCount--;
}

telegramQueue[telegramTail] = msg;
telegramTail = (telegramTail + 1) % TELEGRAM_QUEUE_SIZE;
telegramCount++;
}

bool dequeueTelegramMessage(String &msg) {
    if (telegramCount == 0) return false;
    msg = telegramQueue[telegramHead];
    telegramHead = (telegramHead + 1) % TELEGRAM_QUEUE_SIZE;
    telegramCount--;
    return true;
}

void sendTelegramMessage(const String &text) {
    if (WiFi.status() != WL_CONNECTED) return;

    WiFiClientSecure client;
    client.setInsecure();

    HTTPClient https;
    https.setTimeout(1500);

    String url = "https://api.telegram.org/bot" + String(TELEGRAM_BOT_TOKEN) +
        "/sendMessage?chat_id=" + String(TELEGRAM_CHAT_ID) +
        "&text=" + urlencode(text);

```

```

if (https.begin(client, url)) {
  https.GET();
  https.end();
}
}

```

```

void processTelegramQueue() {
  if (telegramCount == 0) return;
  if (millis() - lastTelegramSend < telegramGapMs) return;
  if (millis() - lastHttpActionTime < httpQuietBeforeTelegramMs) return;

```

```

  String msg;
  if (dequeueTelegramMessage(msg)) {
    sendTelegramMessage(msg);
    lastTelegramSend = millis();
  }
}

```

```

void applyOutputs() {
  digitalWrite(LED_PIN, ledState ? HIGH : LOW);
  digitalWrite(FAN_PIN, fanState ? HIGH : LOW);
}

```

```

float generateRandomTemperature() {
  return 25.0 + (random(0, 100) / 10.0); // 25.0 -> 34.9
}

```

```

float generateRandomHumidity() {
  return 50.0 + (random(0, 300) / 10.0); // 50.0 -> 79.9
}

```

```
String buildStatusJson() {
    String json = "{";
    json += "\"led\":\"" + String(ledState ? "true" : "false") + ",";
    json += "\"fan\":\"" + String(fanState ? "true" : "false") + ",";
    json += "\"temperature\":\"" + String(temperatureC, 1) + ",";
    json += "\"humidity\":\"" + String(humidity, 1) + ",";
    json += "\"ldr\":\"" + String(ldrValue) + ",";
    json += "\"pir\":\"" + String(pirValue) + ",";
    json += "\"lightText\":\"" + String(ldrValue < LDR_DARK_THRESHOLD ? "TOI" :
"SANG") + "\"";
    json += "}";
    return json;
}
```

```
String htmlPage() {
    return R"rawliteral(
<!DOCTYPE html>
<html lang="vi">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>ESP32 Home Control</title>
    <style>
        body { font-family: Arial, sans-serif; background: #f4f6f9; margin: 0; padding: 20px; }
        .box { max-width: 980px; margin: auto; background: white; padding: 20px; border-radius:
16px; box-shadow: 0 4px 16px rgba(0,0,0,0.12); }
        h1 { text-align: center; margin-bottom: 24px; }
        .grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(260px, 1fr)); gap:
16px; }
```

```

.card { padding: 18px; border-radius: 14px; background: #fafafa; border: 1px solid #ddd; }
.btn { display: inline-block; padding: 10px 16px; margin: 6px 6px 0 0; border: none; border-
radius: 10px; color: white; cursor: pointer; font-size: 16px; }
.on { background: #28a745; }
.off { background: #dc3545; }
.refresh { background: #007bff; }
.state-on { color: #28a745; font-weight: bold; }
.state-off { color: #dc3545; font-weight: bold; }
.sensor { font-size: 18px; margin: 8px 0; }
</style>
</head>
<body>
<div class="box">
<h1>Dieu khien thiet bi gia dung qua Wi-Fi voi ESP32</h1>

<div class="grid">
<div class="card">
<h3>Den LED</h3>
<p>Trang thai: <span id="ledState" class="state-off">DANG TAT</span></p>
<button class="btn on" onclick="controlDevice('/api/led/on', 'led', true)">Bat
den</button>
<button class="btn off" onclick="controlDevice('/api/led/off', 'led', false)">Tat
den</button>
</div>

<div class="card">
<h3>Quat</h3>
<p>Trang thai: <span id="fanState" class="state-off">DANG TAT</span></p>
<button class="btn on" onclick="controlDevice('/api/fan/on', 'fan', true)">Bat
quat</button>

```

```

    <button class="btn off" onclick="controlDevice('/api/fan/off', 'fan', false)">Tat
    quat</button>
  </div>

  <div class="card">
    <h3>Nhiet do - Do am (mo phong)</h3>
    <div class="sensor">Nhiet do: <span id="tempText">Dang doc...</span></div>
    <div class="sensor">Do am: <span id="humText">Dang doc...</span></div>
  </div>

  <div class="card">
    <h3>Cac cam bien khac</h3>
    <div class="sensor">PIR: <span id="pirText">Dang doc...</span></div>
    <div class="sensor">LDR: <span id="ldrText">Dang doc...</span></div>
  </div>
</div>

<div style="margin-top:16px;">
  <button class="btn refresh" onclick="loadStatus()">Tai lai trang thai</button>
</div>
</div>

<script>
function setDeviceStateUI(id, isOn) {
  const el = document.getElementById(id === 'led' ? 'ledState' : 'fanState');
  el.textContent = isOn ? 'DANG BAT' : 'DANG TAT';
  el.className = isOn ? 'state-on' : 'state-off';
}

async function fetchWithTimeout(url, ms = 1000) {

```

```

const controller = new AbortController();
const timer = setTimeout(() => controller.abort(), ms);
try {
  return await fetch(url, { signal: controller.signal, cache: 'no-store' });
} finally {
  clearTimeout(timer);
}
}

async function loadStatus() {
  try {
    const res = await fetchWithTimeout('/status', 1000);
    const data = await res.json();

    setDeviceStateUI('led', data.led);
    setDeviceStateUI('fan', data.fan);

    document.getElementById('tempText').innerHTML = data.temperature + ' &deg;C';
    document.getElementById('humText').textContent = data.humidity + ' %';
    document.getElementById('pirText').textContent = data.pir ? 'CO CHUYEN DONG' :
'KHONG CO';
    document.getElementById('ldrText').textContent = data.ldr + ' (' + data.lightText + ')';
  } catch (e) {
    console.log('Khong tai duoc /status', e);
  }
}

async function controlDevice(url, id, isOn) {
  setDeviceStateUI(id, isOn);
  try {

```

```

    await fetchWithTimeout(url, 500);
    setTimeout(loadStatus, 100);
  } catch (e) {
    console.log('Khong goi duoc lenh dieu khien', e);
  }
}

```

```

loadStatus();
setInterval(loadStatus, 8000);
</script>
</body>
</html>
)rawliteral";
}

```

```

void handleRoot() {
  server.send(200, "text/html; charset=UTF-8", htmlPage());
}

```

```

void handleLedOn() {
  ledState = true;
  applyOutputs();
  lastHttpActionTime = millis();
  enqueueTelegramMessage("Thong bao: Den LED da BAT");
  server.send(200, "application/json", "{\"ok\":true,\"device\":\"led\",\"state\":true}");
}

```

```

void handleLedOff() {
  ledState = false;
  applyOutputs();
}

```

```

lastHttpActionTime = millis();
enqueueTelegramMessage("Thong bao: Den LED da TAT");
server.send(200, "application/json", "{\"ok\":true,\"device\":\"led\",\"state\":false}");
}

```

```

void handleFanOn() {
    fanState = true;
    applyOutputs();
    lastHttpActionTime = millis();
    enqueueTelegramMessage("Thong bao: Quat da BAT");
    server.send(200, "application/json", "{\"ok\":true,\"device\":\"fan\",\"state\":true}");
}

```

```

void handleFanOff() {
    fanState = false;
    applyOutputs();
    lastHttpActionTime = millis();
    enqueueTelegramMessage("Thong bao: Quat da TAT");
    server.send(200, "application/json", "{\"ok\":true,\"device\":\"fan\",\"state\":false}");
}

```

```

void handleStatusJson() {
    server.send(200, "application/json", buildStatusJson());
}

```

```

void readSensors() {
    ldrValue = analogRead(LDR_PIN);
    pirValue = digitalRead(PIR_PIN);

```

```

    temperatureC = generateRandomTemperature();

```

```

    humidity = generateRandomHumidity();
}

void checkAlerts() {
    unsigned long now = millis();

    if (temperatureC > TEMP_ALERT_THRESHOLD &&
        (now - lastTempAlertTime > ALERT_COOLDOWN_MS)) {
        enqueueTelegramMessage("CANH BAO: Nhiet do vuot nguong! Nhiet do hien tai: " +
String(temperatureC, 1) + " *C");
        lastTempAlertTime = now;
    }

    if (pirValue == HIGH &&
        (now - lastMotionAlertTime > ALERT_COOLDOWN_MS)) {
        enqueueTelegramMessage("CANH BAO: Phat hien chuyen dong trong khu vuc giam sat");
        lastMotionAlertTime = now;
    }
}

void setupWebServer() {
    server.on("/", handleRoot);
    server.on("/api/led/on", handleLedOn);
    server.on("/api/led/off", handleLedOff);
    server.on("/api/fan/on", handleFanOn);
    server.on("/api/fan/off", handleFanOff);
    server.on("/status", handleStatusJson);
    server.begin();
}

```

```

void connectWiFi() {
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD, 6);
  while (WiFi.status() != WL_CONNECTED) {
    delay(200);
  }
}

```

```

void setup() {
  Serial.begin(115200);
  randomSeed(micros());

  pinMode(LED_PIN, OUTPUT);
  pinMode(FAN_PIN, OUTPUT);
  pinMode(PIR_PIN, INPUT);

```

```

  applyOutputs();
  connectWiFi();
  setupWebServer();
  readSensors();

```

```

  enqueueTelegramMessage("ESP32 da khoi dong thanh cong. He thong HTTP dieu khien da
san sang.");
}

```

```

void loop() {
  server.handleClient();
  processTelegramQueue();

  unsigned long now = millis();
  if (now - lastSensorRead >= sensorInterval) {

```

```

    lastSensorRead = now;
    readSensors();
    checkAlerts();
}
}

```

Mã nguồn file config.h:

```

#pragma once

#define WIFI_SSID    "Wokwi-GUEST"
#define WIFI_PASSWORD ""

// DOI TOKEN MOI CUA BAN VAO DAY
#define TELEGRAM_BOT_TOKEN
"8700950437:AAE_jVD1ng9j063_HRczki3Ky19e1dByArE"
#define TELEGRAM_CHAT_ID "8297209145"

#define LED_PIN  25
#define FAN_PIN  26
#define PIR_PIN  13
#define LDR_PIN  34

#define TEMP_ALERT_THRESHOLD  35.0
#define LDR_DARK_THRESHOLD    1500
#define ALERT_COOLDOWN_MS     60000

```

Mã nguồn file Diagram.json:

```

{
  "version": 1,
  "author": "test",

```

```

"editor": "wokwi",
"parts": [
  { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 23.9, "left": 23.8, "attrs": { } },

  { "type": "wokwi-led", "id": "led1", "top": -42, "left": -188.2, "attrs": { "color": "red" } },
  { "type": "wokwi-led", "id": "led2", "top": -61.2, "left": 147.8, "attrs": { "color": "limegreen"
} },

  { "type": "wokwi-resistor", "id": "r1", "top": -5.65, "left": -67.2, "attrs": { "value": "220" } },
  { "type": "wokwi-resistor", "id": "r2", "top": 3.95, "left": 172.8, "attrs": { "value": "220" } },

  { "type": "wokwi-pir-motion-sensor", "id": "pir1", "top": -215, "left": 210, "attrs": { } },
  { "type": "wokwi-photoresistor-sensor", "id": "ldr1", "top": 245, "left": 220, "attrs": { } }
],
"connections": [
  [ "esp:D25", "r1:2", "green", [ "h0" ] ],
  [ "r1:1", "led1:A", "green", [ "h-96" ] ],
  [ "led1:C", "esp:GND.2", "green", [ "v0" ] ],

  [ "esp:D26", "r2:1", "green", [ "h-28.8", "v-115.3" ] ],
  [ "r2:2", "led2:A", "green", [ "v0" ] ],
  [ "led2:C", "esp:GND.1", "green", [ "v0" ] ],

  [ "esp:5V", "pir1:VCC", "red", [ "v-20", "h130" ] ],
  [ "esp:D13", "pir1:OUT", "blue", [ "v-10", "h170" ] ],
  [ "esp:GND.2", "pir1:GND", "black", [ "v-30", "h190" ] ],

  [ "esp:3V3", "ldr1:VCC", "red", [ "v40", "h180" ] ],
  [ "esp:34", "ldr1:AO", "orange", [ "v30", "h210" ] ],
  [ "esp:GND.1", "ldr1:GND", "black", [ "v50", "h160" ] ]

```

```
],  
  "dependencies": {}  
}
```

PHIẾU ĐÁNH GIÁ TIỂU LUẬN

Học kỳ Năm học ...-....

Cán bộ chấm thi 1	Cán bộ chấm thi 2
Nhận xét:	Nhận xét:
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
Điểm đánh giá của CBChT1:	Điểm đánh giá của CBChT2:
Bằng số:	Bằng số:
Bằng chữ:	Bằng chữ:

Điểm kết luận: Bằng số..... Bằng chữ:.....

Thành phố Huế, ngày tháng năm 20...

CBChT1

(Ký và ghi rõ họ tên)

CBChT2

(Ký và ghi rõ họ tên)